

HastyCR — Clash Royale Bot

Agent handoff after an unattended overnight run.

Run window: 2026-08-16, 02:51 → 11:15 · 36 blocks · 168 matches · 3,719 cards played · 9,796 frames captured

The record: 0 wins, 168 losses

Read from 168 end-of-match result screenshots by `scripts/results.py`, which is independent of tower-HP detection and was validated against three visually confirmed screens. The bot plays autonomously and competently by several behavioural measures, and it has **never won a match**.

Do not trust any crown/tower statistic in the older logs. `towers=` on `MATCH_RESULT` is sampled from the final in-game frame, which lands inside the end-of-match sequence when HP bars vanish, so towers read `0.00` whether or not they fell. A match logged as "enemy 0.00/0.00, 2 crowns for us" was in fact a 3-0 defeat.

What to trust

Signal	Source	Status
Win / loss outcome	<code>scripts/results.py</code> (result screen banner position)	RELIABLE — validated on 3 screens
Push conversion	<code>scripts/conversion.py</code> (mid-match enemy HP from PLAY lines)	RELIABLE — mid-match HP validated same-frame
Card usage, hog share, shim counts	<code>scripts/analyze_run.py</code> , block logs	RELIABLE — hundreds of events/block
Crowns for / against	<code>scripts/trend.py</code> (<code>MATCH_RESULT</code> towers)	VOID — see alert above
Ally tower HP	vendored detector, and <code>scripts/tower_hp.py</code>	BROKEN in both — see Known bugs

Verified results from the run

Attacking improved measurably and did not produce a single win.

- Hog Rider plays: **0 in the first 30 cards** → **3.7 per match** (the elixir ceiling is ~3–4/match)
- Push conversion: **60%** → **76%** pooled over 119 pushes; recent blocks 76–83%
- Damage output: **~0.75** → **~1.0 enemy tower per match**
- Cannon placement tightened from x 8–13 into the x 8–10, y 20–22 pocket
- Spells stopped being thrown at empty tiles (retarget onto densest cluster; 33 wasted casts → ~7/block)

Conclusion: defence is the entire remaining gap — and defence is exactly what cannot be measured until ally tower HP detection is fixed.

Architecture

Component	Role
vendor/ClashRoyaleBuildABot	Perception. <code>Detector.run(frame)</code> → screen name, elixir, hand cards, tower HP, unit positions.
vendor/KataCR	Policy. <code>StARformer_3L</code> offline-RL checkpoint (epoch 003), wrapped by <code>src/hastycr/adapters/katacr.py</code> .
scripts/mumu_katacr.py	The runner: perception → policy → shims → guarded ADB taps. All flags live here.
scripts/policy_shims.py	Hand-written corrections layered on the model's output (the active strategy layer).
scripts/heuristic_policy.py	Full hand-written 2.6 policy. Available via <code>--heuristic</code> ; currently OFF (performed worse).
Control path	Raw ADB against MuMu at <code>127.0.0.1:7555</code> , 1080×1920. <code>exec-out</code> <code>screencap</code> in, <code>input tap</code> out.

The model emits `(slot, x, y, delay)` on an 18×32 top-down grid ($y < 16$ enemy half, $y \geq 16$ ours); `grid_to_mumu()` maps that to pixels.

Running it

```
cd C:\Users\aksha\Downloads\HastyCR
& "C:\Program Files\Netease\MuMuPlayer\nx_device\15.0\shell\adb.exe" connect 127.0.0.1:7555

# single supervised run (5 matches, then exits)
.\venvs\buildabot\Scripts\python.exe .\scripts\mumu_katacr.py `
  --adb "C:\Program Files\Netease\MuMuPlayer\nx_device\15.0\shell\adb.exe" `
  --serial 127.0.0.1:7555 --epoch 3 --hours 1 --auto-queue --shims --max-matches 5

# unattended: 5-match blocks forever, auto-analysis, capture daemon, self-restart
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\overnight_supervisor.ps1
```

Two traps that cost hours during this run. (1) PowerShell parses `overnight_supervisor.ps1` at launch — editing it does nothing until the supervisor is restarted, and the running bot will silently keep the old flags. Verify with the process command line, not the file. (2) `.py` files reload per block, so Python edits take effect at the next block without a restart.

Analysis tools

```
.\venvs\buildabot\Scripts\python.exe scripts\results.py tmp\live\blocks\katacr_results # true W/L
.\venvs\buildabot\Scripts\python.exe scripts\conversion.py --per-block # push
conversion
.\venvs\buildabot\Scripts\python.exe scripts\trend.py # per-block
behaviour
.\venvs\buildabot\Scripts\python.exe scripts\analyze_run.py tmp\live\blocks\block_NNN.log
```

Result screenshots live in `tmp/live/blocks/katacr_results/` (the runner derives the folder from `--log` 's parent). `tmp/live/katacr_results/` holds a previous session.

Known bugs — day-2 priorities

1. **Ally tower HP detection is broken (highest priority).** The vendored detector reports our right princess tower at `0.00` while it sits at full 3346 HP — reproduced on multiple identical frames. A replacement bar reader (`scripts/tower_hp.py`) matched ground truth on two frames and failed on a third because *the HP bar's vertical position shifts between matches* (y 1177–1194 vs y 1160–1177). Widening the search band made it worse (read 0.966 for a true 0.369) by catching other blue UI. *Suggested fix: anchor the ROI to a detected landmark — the tower sprite or level badge — instead of absolute pixels.* Until this is fixed, defensive performance cannot be measured at all.
2. **End-of-match tower sampling.** `last_towers` is captured on the final in-game frame, inside the end-of-match animation. Sample it a second or two earlier, or take the value from the last frame where `battle_guard` passed with bars present.
3. `pre_card_guard` **rejects ~149 decisions/night.** The hand shifts between the model's decision frame (~1s stale) and the tap. The guard is correct and must not be weakened — it prevents playing the wrong card — but decision latency is the underlying cost.
4. **Model emits out-of-arena coordinates** (observed `(-2,28)` , `(-2,33)`). Now clamped and logged as `CLAMP_OOB` rather than raising.

Strategy layer (`policy_shims.py`)

Shim	What it does	Evidence
<code>force_hog</code>	Plays Hog at the bridge when affordable and nothing serious is on our half	Model played it 0 times in 30 cards; now 3.7/match
<code>hog_lane</code>	Alternates lanes when enemy towers are equal, commits when one is ≥ 0.10 weaker	Fixed a bias that sent every Hog left and fed our left tower to counter-pushes
<code>defend_*</code>	Swaps chip cards for Musketeer/Cannon/Ice Golem under real pressure	Triggers on ≥ 2 units or any unit at $y \geq 20$
<code>clamp_building</code>	Snaps Cannon into x 8–10, y 20–22	Placement spread was x 8–13
<code>retarget_*</code>	Moves a spell onto the densest enemy cluster; vetoes if none	Wasted casts 33/block $\rightarrow$ ~7/block
<code>finish_*</code>	Fireballs an enemy tower already ≤ 0.18 HP	Converted a 0.18 tower into a kill
<code>support_*</code> , <code>tank_first</code>	Escorts the Hog (tank ahead, support behind)	<code>tank_first</code> rarely fires — Ice Golem is usually spent on defence first

Safety layer — do not weaken

- `battle_guard` / `validate_battle` gate every card tap on a confirmed live battle (two live tower bars, ≥ 3 real cards in hand), re-checked before selecting a card *and* before placing it.
- The runner never taps an unrecognised screen except a positively identified popup close button (`popup_guard.py` : colour + geometry + shape, validated against 172 archived screens).

- Nothing taps shop, gem, or purchase controls. A Pass Royale offer popup was dismissed via its X during the run and the gem balance was verified unchanged at 990.
- `RECOVER` relauches Clash Royale if the screen stays unknown for 3 minutes (rate-limited to once per 5 minutes).

Methodology warnings

Outcomes are statistically underpowered at this cadence. Crowns arrived at ~0.7 per 5-match block; detecting even a 50% improvement needs roughly **80–95 blocks (400–475 matches) per configuration**. The entire run was 36 blocks. Behaviour metrics (card counts, placement spread, conversion) are reliable because they count hundreds of events per block; win-rate comparisons across a few blocks are not.

Things that looked true and were not, during this run: that the hand-written policy beat the model (3 blocks of data, noise); that enemy tower HP was unreliable (compared two captures taken seconds apart — it is accurate mid-match); that a fast match was an opponent walkover (it was a 52-second 3-0 defeat). Validate on identical frames, and prefer `results.py` over derived metrics.

What was already tried

- **Hand-written policy driving decisions** (`--heuristic`): scored worse than model+shims and turtled badly (194 defends vs 9 pushes in one block) before the gates were corrected. Left in the tree, disabled.
- **Raising Hog frequency:** done and capped. ~64 elixir/match at 2.6 avg cost $\approx$ 24–32 cards; Hog is 1 of 8 and only reachable by playing the other three, so ~3–4/match is the physical ceiling. The bot is at it. *More Hog is not available.*
- **Replacing tower-HP perception:** attempted and reverted (see Known bugs #1).

Suggested starting point

1. Fix ally tower HP (landmark-anchored ROI), then re-measure defence — it is the only unmeasured half of the game.
2. Use `results.py` as the sole success metric. Target: **a first win**, not crown counts.
3. If the policy is the ceiling, note the checkpoint (`StARformer_3L_v0.8_golem_ai`) was trained against KataCR's scripted golem bot, not 5500-trophy ladder opponents — it is off-distribution by construction.

Full chronological iteration log, including every wrong turn and its correction: `docs/OVERNIGHT_HANDOFF.md`.